Compiler Design Lab Report 1
Lexical Analysis Implementation

Abdullah Al Jubaer Gem
ID: 210041226
Section: 2B

May 7, 2026

1 Introduction

The objective of this lab is to implement a Lexical Analyzer (Scanner) that reads a source program and identifies valid tokens such as keywords, identifiers, numbers, operators, and delimiters.

2 Token Definitions

The following categories of tokens are defined in the lexical analyzer:

- **Keywords:** Reserved words used by the language syntax.
  `int, float, if, else, while, for, return, var`
- **Identifiers:** Names for variables, functions, etc. Defined by the regex:
  `[a-zA-Z_][a-zA-Z0-9_]*`
- **Numbers:**
  - **Integers:** `[0-9]+`
  - **Floats:** `[0-9]+\.[0-9]+`
- **Operators:** Mathematical and logical symbols.
  `+, -, *, /, =, ==, !=, <, >, <=, >=`
- **Delimiters:** Punctuation marks used for structural grouping.
  `(, ), {, }, ;, ,`
- **Whitespaces:** Characters like spaces, tabs, and newlines are used as separators but are not treated as tokens.

3 Solution Approach

The lexical analyzer follows a character-by-character scanning approach:

1. **File Reading:** The input file is opened and read one character at a time.
2. **Lexeme Formation:** Characters are accumulated into a buffer (lexeme) until a breakpoint (delimiter, operator, or whitespace) is encountered.
3. **Look-ahead Logic:** For multi-character operators (e.g., `==` vs `=`), the scanner performs a look-ahead to check if the next character forms a valid combined operator.
4. **Token Categorization:**
   - Keywords are checked against a predefined list.
   - Identifiers and numbers are validated using Regular Expressions (Regex).

- Operators and delimiters are matched directly against predefined sets.

5. **Error Handling:** Any sequence of characters that does not match the predefined patterns is categorized as an `INVALID` token.

4 Implementation

The implementation is done in Python. The source code is provided below:

```
import re

keywords = ["int", "float", "if", "else", "while", "for", "return", "var"]
whitespace = [" ", "\n", "\t"]
delimiters = ["(", ")", "{", "}", ";", ","]
operators = ["+", "-", "*", "/", "=", "==", "!=", "<", ">", "<=", ">="]

identifier_pattern = r"[a-zA-Z_][a-zA-Z0-9_]*"
integer_pattern = r"[0-9]+"
float_pattern = r"[0-9]+\.[0-9]+"

def is_keyword(token):
    return token in keywords

def is_delimiter(token):
    return token in delimiters

def is_operator(token):
    return token in operators

def is_identifier(token):
    return re.fullmatch(identifier_pattern, token) is not None

def is_integer(token):
    return re.fullmatch(integer_pattern, token) is not None

def is_float(token):
    return re.fullmatch(float_pattern, token) is not None

def print_token(token_type, lexeme):
    print(f"<{token_type}, {lexeme}>")

def identify_token(lexeme):
    if is_keyword(lexeme):
        print_token("KEYWORD", lexeme)
    elif is_float(lexeme):
        print_token("NUMBER", lexeme)
    elif is_integer(lexeme):
        print_token("NUMBER", lexeme)
    elif is_identifier(lexeme):
        print_token("IDENTIFIER", lexeme)
    else:
        print_token("INVALID", lexeme)

def lexical_analyzer(filename):
    lexeme = ""
    pending_char = ""

    with open(filename, "r") as file:
        while True:
            if pending_char != "":
                char = pending_char
                pending_char = ""
            else:
                char = file.read(1)

            if char == "":
                if lexeme != "":
                    identify_token(lexeme)
                break

            if char in whitespace:
                if lexeme != "":
```

```
                    identify_token(lexeme)
                    lexeme = ""
                continue

            if char in delimiters:
                if lexeme != "":
                    identify_token(lexeme)
                    lexeme = ""
                print_token("DELIMITER", char)
                continue

            if char in operators:
                if lexeme != "":
                    identify_token(lexeme)
                    lexeme = ""

                next_char = file.read(1)
                possible_operator = char + next_char

                if possible_operator in operators:
                    print_token("OPERATOR", possible_operator)
                else:
                    if char in operators:
                        print_token("OPERATOR", char)
                    else:
                        print_token("INVALID", char)
                    pending_char = next_char
                continue

            if char.isalnum() or char == "_" or char == ".":
                lexeme += char
                continue

            if lexeme != "":
                identify_token(lexeme)
                lexeme = ""

            print_token("INVALID", char)

if __name__ == "__main__":
    lexical_analyzer("input.txt")
```

Listing 1: Lexical Analyzer Source Code

5 Sample Input and Output

5.1 Input (input.txt)

```
int a = 10
float b = a + 10
var x = (b / 3)
```

5.2 Output Log

```
<KEYWORD, int>
<IDENTIFIER, a>
<OPERATOR, =>
<NUMBER, 10>
<KEYWORD, float>
<IDENTIFIER, b>
<OPERATOR, =>
<IDENTIFIER, a>
<OPERATOR, +>
<NUMBER, 10>
<KEYWORD, var>
<IDENTIFIER, x>
<OPERATOR, =>
<DELIMITER, (>
<IDENTIFIER, b>
```

```
<OPERATOR, />
<NUMBER, 3>
<DELIMITER, )>
```

6 Conclusion

The lexical analyzer successfully correctly identifies and tokenizes the given input program according to the specified rules. It handles keywords, identifiers, numbers (both integer and float), operators (including multi-character ones), and delimiters while effectively ignoring whitespaces.